

GhostGraph

AI-Assisted Security Scanner Prototype

An experimental tool that uses AI to filter out noise from traditional scanners.

The Problem: Alert Fatigue

Modern SAST tools generate thousands of alerts. Most of them are noise.

High False Positive Rates

Traditional scanners flag test files, demo secrets, and unreachable dead code as critical vulnerabilities.

Developer Friction

Security engineers spend roughly 80% of their time triaging noise instead of fixing the real critical bugs.

No Business Context

A rule flags a raw SQL query, but has no way to know the input was already sanitized 3 functions upstream.

Missed Real Threats

While teams drown in noise, the real exploitable vulnerabilities slip through unnoticed.

The Solution: AI-Assisted Threat Hunting

GhostGraph uses local LLMs to help filter alerts and identify real vulnerabilities.

1. SCAN

Upload a ZIP. The engine extracts source code and runs Semgrep (SAST) + npm audit (SCA) deterministic rules.

2. THINK

Each flagged code snippet is sent with a 250-line context window to a local LLM (Llama 3 via Ollama).

3. FILTER

The AI analyzes reachability and context. Test data, demo secrets, and dead code are auto-dismissed as noise.

4. REMEDIATE

Verified vulnerabilities get an executive summary, business impact analysis, and a secure code patch.

System Architecture

FRONTEND	BACKEND	AI / DATA
React + TypeScript	FastAPI (Python)	Ollama (Local LLM)
Vite Dev Server	SQLModel + SQLite	Semgrep Engine
Recharts Analytics	Uvicorn ASGI	npm audit (SCA)
WebSocket Terminal	Background Workers	ChromaDB Vectors
Tailwind CSS	WebSocket Manager	SentenceTransformers

All AI processing runs 100% locally via Ollama. Zero source code ever leaves the network. Air-gap ready.

Live Telemetry & Threat Intel

> Engine Logs (Live)

```
[14:32:01] uvicorn.access: POST /api/engagements/3/scan 200
[14:32:01] sqlalchemy: SELECT engagement.id FROM engagement
[14:32:02] ghostgraph: Extracting source code from app.zip...
[14:32:05] ghostgraph: Launching Semgrep SAST scan...
[14:32:08] ghostgraph: Running SCA via npm audit...
[14:32:12] ghostgraph: 47 raw findings detected.
[14:32:12] ghostgraph: De-duplicating... 31 unique findings.
[14:32:13] ghostgraph: AI Pre-Filter started on 31 findings.
[14:32:18] ollama: Finding 1/31 - False Positive (test data)
[14:32:22] ollama: Finding 2/31 - TRUE POSITIVE (SQLi)
[14:32:25] ghostgraph: Background filtering 35% complete...
```

Real-time WebSocket intercept of Python logging

Threat Intelligence (RAG)

Built-in ChromaDB vector database stores security TTPs. Users search by concept (e.g., 'Reflected XSS') and the engine performs semantic nearest-neighbor retrieval using sentence-transformer embeddings.

Air-Gapped & Private

All LLM inference runs on-premises via Ollama. No API keys. No cloud calls. Your proprietary source code never leaves the machine. Perfect for regulated industries and red team operations.

Exactly How It Works (Data Flow)

1

Upload

User uploads a .zip via the React frontend. Backend creates a temp directory and extracts the source code.

2

Scan

Backend spawns async subprocesses: Semgrep (OWASP Top 10, CWE-25 rulesets) and npm audit (dependency CVEs).

3

De-Duplicate

Findings are sorted by file path + line number. Alerts within 5 lines of each other are merged into one.

4

AI Pre-Filter

A background worker sends each finding with a 250-line context window to Ollama. The LLM replies in strict JSON: `is_real_vulnerability` (bool) + `reason`. False positives are auto-dismissed.

5

Deep Analysis

User clicks 'Ask GhostGraph AI' on a verified finding. Ollama generates: Explanation, Business Impact, and a Secure Code Patch.

6

Report

Results are stored in SQLite. User can export a full PDF report or browse the interactive dashboard.

What We Used & Why (Plain English)

- **React** The user interface you see in the browser. It draws the buttons, tables, and charts.
- **FastAPI** The Python web server that handles all requests behind the scenes.
- **Semgrep** An open-source code scanner. It reads source files and matches patterns for common bugs.
- **npm audit** Checks if any libraries the project depends on have known security holes (CVEs).
- **Ollama** Runs AI language models (like Llama 3) entirely on your own machine. No cloud needed.
- **ChromaDB** A small local database that stores security knowledge as math vectors for smart search.
- **SQLite** A tiny file-based database that stores your projects, findings, and scan history.
- **WebSockets** A live two-way connection so the terminal updates instantly without refreshing.

How We Built It (Step by Step)

1

Built the Backend

We wrote a Python server using FastAPI. It handles file uploads, runs scans, stores results in a SQLite database, and talks to the AI.

2

Built the Frontend

We created a React app with TypeScript. It shows the dashboard, findings list, charts, and a live terminal. It talks to the backend over HTTP.

3

Plugged in Semgrep

We added Semgrep as our main code scanner. It uses open-source security rulesets (OWASP Top 10, CWE Top 25) to find vulnerabilities.

4

Added AI Filtering

We connected to Ollama (a local AI server). After Semgrep flags code, we send 250 lines of context to the AI and ask: is this real or noise?

5

Added Threat Intel

We set up ChromaDB to store security knowledge. Users can search concepts like 'SQL Injection' and get smart results even without exact keywords.

6

Wired Up Live Logs

We hooked Python's logging system into a WebSocket so every database query and API call streams to the frontend terminal in real time.

Example: Real Vulnerability Found

Semgrep flags it. The AI confirms it. GhostGraph gives you the fix.

VULNERABLE CODE (app/routes/users.py:34)

```
def get_user(request):  
    user_id = request.args.get("id")  
    query = f"SELECT * FROM users  
            WHERE id = {user_id}"  
    result = db.execute(query)  
    return jsonify(result)
```

AI VERDICT: TRUE POSITIVE

User input is directly interpolated into a SQL query.

SECURE FIX (AI-Generated Patch)

```
def get_user(request):  
    user_id = request.args.get("id")  
    query = "SELECT * FROM users  
            WHERE id = ?"  
    result = db.execute(query,  
                        (user_id,))  
    return jsonify(result)
```

PARAMETERIZED QUERY - SAFE

Business Impact (AI-Generated):

An attacker can manipulate the 'id' parameter to dump the entire users table, bypass authentication, or delete records. This is a classic SQL Injection (CWE-89) and is rated CVSS 9.8 Critical.

Example: False Positive Dismissed

Traditional scanners would flag this. GhostGraph's AI knows better.

FLAGGED CODE (tests/test_auth.py:12) - Semgrep Rule: hardcoded-credentials

```
# test_auth.py - Unit tests for login flow

class TestAuthentication(unittest.TestCase):

    def test_login_success(self):

        test_password = "P@ssw0rd123"      # <-- flagged

        response = self.client.post("/login",

            json={"user": "test", "pass": test_password})

        self.assertEqual(response.status_code, 200)
```

TRADITIONAL SCANNER RESULT

CRITICAL: Hardcoded credentials detected!

You waste 15 minutes investigating a test file.

GHOSTGRAPH AI RESULT

FALSE POSITIVE - Auto-dismissed.

AI Reason: This is test data in a unit test file. Not production code.

GhostGraph

An AI-Assisted Security Scanner Prototype

100%

Local & Private

80%+

Noise Reduction

0

Cloud API Calls

Built with FastAPI + React + Ollama + Semgrep + ChromaDB